EX. NO: 6 **DOUBLY LINKED LISTS**

DATE:

QUESTION :

Implement creation of a sorted linked list, Deletion of a node, search a value, merge two sorted lists, and display operations on Doubly Linked List

AIM:

To Implement creation of a sorted linked list, Deletion of a node, search a value, merge two sorted lists, and display operations on Doubly Linked List

ALGORITHM:

Step-1. Start
Step-2. Define a structure Node containing:
- data to store the value.
- prev pointer to point to the previous node.
- next pointer to point to the next node.

Step-3. Initialize the head pointer as NULL.
Step-4. For insertion at the beginning:
- Create a new node using malloc().
- Store the value in the node.
- Set its prev as NULL and next as head.
- If the list is not empty, update the previous pointer of the old head.
- Make the new node as head.

Step-5. For insertion in the middle:
- Traverse the list to find the given key value.
- Create a new node and store the value.
- Connect the new node between the current node and the next node.
- Update the prev and next pointers accordingly.

Step-6. For insertion at the end:
- Create a new node and set its next as NULL.
- Traverse to the last node.
- Link the last node to the new node and set the new node's prev.

Step-7. For deletion at the beginning:
- Check whether the list is empty.
- Move head to the next node.
- Set the new head's prev as NULL.
- Free the deleted node.

Step-8. For deletion in the middle:
- Traverse the list and find the node containing the given value.
- Connect its previous node to its next node.
- Update the prev pointer of the next node.
- Free the deleted node.

Step-9. For deletion at the end:
- Traverse to the last node.
- Set the previous node's next as NULL.
- Free the last node.

Step-10 For searching:
- Traverse the list from head.
- Compare each node's data with the given search value.
- If found, display its position; otherwise display "Element not found."

Step-11 For displaying:
- Check whether the list is empty.
- Traverse from head to NULL.
- Display the data of each node.

Step-12 Use a menu-driven program to perform the required operations repeatedly.
Step-13 Continue until the user selects Exit.
Step-14 Stop

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
   int data;
   struct Node *prev;
   struct Node *next;
};
struct Node *head = NULL;
void insertBeginning(int value) {
   struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->prev = NULL;
   newNode->next = head;
   if (head != NULL)
     head->prev = newNode;
   head = newNode;
}
void insertEnd(int value) {
   struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->next = NULL;
   if (head == NULL) {
      newNode->prev = NULL;
      head = newNode;
      return;
   }
   struct Node *temp = head;
   while (temp->next != NULL)
      temp = temp->next;
   temp->next = newNode;
   newNode->prev = temp;
}

void insertMiddle(int key, int value) {
   struct Node *temp = head;
   while (temp != NULL && temp->data != key)
      temp = temp->next;
```

```
    if (temp == NULL) {
        printf("Node not found.\n");
        return;
    }
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL)
        temp->next->prev = newNode;

    temp->next = newNode;
}
void deleteBeginning() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    if (head != NULL)
        head->prev = NULL;
    free(temp);
    printf("First node deleted.\n");
}
void deleteEnd() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    if (head->next == NULL) {
        free(head);
        head = NULL;
        printf("Last node deleted.\n");
        return;
    }
    struct Node *temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->prev->next = NULL;
    free(temp);
    printf("Last node deleted.\n");
}
void deleteMiddle(int key) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct Node *temp = head;
    while (temp != NULL && temp->data != key)
        temp = temp->next;
    if (temp == NULL) {
        printf("Node not found.\n");
```

```
        return;
    }
    if (temp == head) {
        deleteBeginning();
        return;
    }
    if (temp->next == NULL) {
        deleteEnd();
        return;
    }
    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;
    free(temp);
    printf("Node deleted.\n");
}
void search(int key) {
    struct Node *temp = head;
    int pos = 1;
    while (temp != NULL) {
        if (temp->data == key) {
            printf("Element found at position %d\n", pos);
            return;
        }
        temp = temp->next;
        pos++;
    }
    printf("Element not found.\n");
}

void display() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct Node *temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
int main() {
    int choice, value, key;

    while (1) {
        printf("\n===== DOUBLY LINKED LIST =====\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at Middle\n");
        printf("3. Insert at End\n");
        printf("4. Delete from Beginning\n");
        printf("5. Delete from Middle\n");
        printf("6. Delete from End\n");
        printf("7. Search\n");
```

```
printf("8. Display\n");
printf("9. Exit\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
   case 1:
      printf("Enter value: ");
      scanf("%d", &value);
      insertBeginning(value);
      break;

   case 2:
      printf("Insert after which value? ");
      scanf("%d", &key);
      printf("Enter new value: ");
      scanf("%d", &value);
      insertMiddle(key, value);
      break;

   case 3:
      printf("Enter value: ");
      scanf("%d", &value);
      insertEnd(value);
      break;

   case 4:
      deleteBeginning();
      break;

   case 5:
      printf("Enter value to delete: ");
      scanf("%d", &key);
      deleteMiddle(key);
      break;

   case 6:
      deleteEnd();
      break;

   case 7:
      printf("Enter value to search: ");
      scanf("%d", &key);
      search(key);
      break;

   case 8:
      display();
      break;

   case 9:
      printf("Exiting...\n");
      exit(0);
```

```
            default:
                printf("Invalid choice.\n");
        }
    }

    return 0;
}
```

INPUT AND OUTPUT:

```
[ita2536@mritlin harshan]$ vi ex6.c
[ita2536@mritlin harshan]$ cc ex6.c
[ita2536@mritlin harshan]$ ./a.out
===== DOUBLY LINKED LIST =====
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete from Beginning
5. Delete from Middle
6. Delete from End
7. Search
8. Display
9. Exit
Enter your choice: 1
Enter value: 2

===== DOUBLY LINKED LIST =====
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete from Beginning
5. Delete from Middle
6. Delete from End
7. Search
8. Display
9. Exit
Enter your choice: 2
Insert after which value? 2
Enter new value: 2

===== DOUBLY LINKED LIST =====
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete from Beginning
5. Delete from Middle
6. Delete from End
7. Search
8. Display
9. Exit
Enter your choice: 3
Enter value: 3
```

```
===== DOUBLY LINKED LIST =====
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete from Beginning
5. Delete from Middle
6. Delete from End
7. Search
8. Display
9. Exit
Enter your choice: 4
First node deleted.

===== DOUBLY LINKED LIST =====
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete from Beginning
5. Delete from Middle
6. Delete from End
7. Search
8. Display
9. Exit
Enter your choice: 1
Enter value: 5

===== DOUBLY LINKED LIST =====
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete from Beginning
5. Delete from Middle
6. Delete from End
7. Search
8. Display
9. Exit
Enter your choice: 1
Enter value: 8
```

```
===== DOUBLY LINKED LIST =====
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete from Beginning
5. Delete from Middle
6. Delete from End
7. Search
8. Display
9. Exit
Enter your choice: 5
Enter value to delete: 2
Node deleted.

===== DOUBLY LINKED LIST =====
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete from Beginning
5. Delete from Middle
6. Delete from End
7. Search
8. Display
9. Exit
Enter your choice: 6
Last node deleted.

===== DOUBLY LINKED LIST =====
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete from Beginning
5. Delete from Middle
6. Delete from End
7. Search
8. Display
9. Exit
Enter your choice: 1
Enter value: 9

===== DOUBLY LINKED LIST =====
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete from Beginning
5. Delete from Middle
6. Delete from End
7. Search
```

```
7. Search
8. Display
9. Exit
Enter your choice: 7
Enter value to search: 4
Element not found.

===== DOUBLY LINKED LIST =====
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete from Beginning
5. Delete from Middle
6. Delete from End
7. Search
8. Display
9. Exit
Enter your choice: 8
Doubly Linked List: 9 8 5

===== DOUBLY LINKED LIST =====
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete from Beginning
5. Delete from Middle
6. Delete from End
7. Search
8. Display
9. Exit
Enter your choice: 9
Exiting...
[ita25536@mritlin harshan]$
```

RESULT:

Thus, the implementation of creation of a sorted linked list, Deletion of a node, search a value, merge two sorted lists, and display operations on Doubly Linked List was executed successfully.